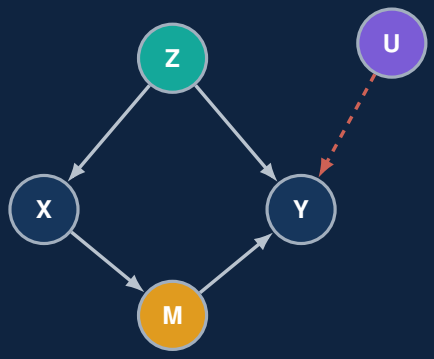


ADVANCED ARTIFICIAL INTELLIGENCE

Probabilistic Graphical Models and Causal Inference

Session 6

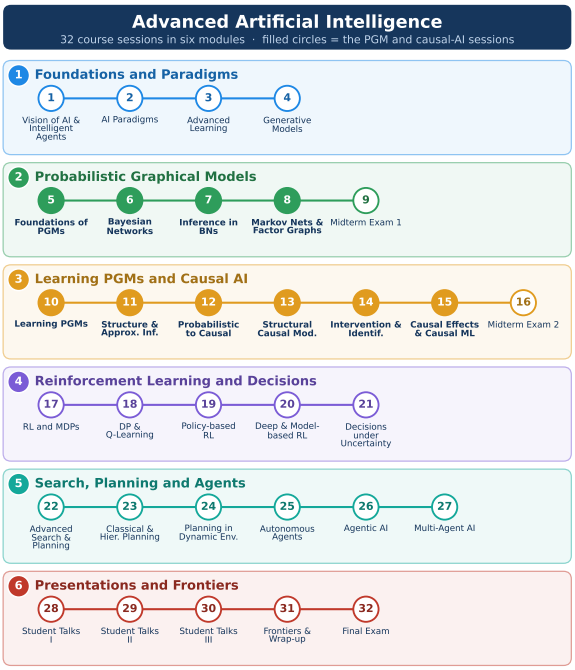
Bayesian Networks: DAGs and Independence Encoding

Lecture Notes

Instructor: Prof. M. Arashi

About these notes

These notes cover Session 6 of the course *Advanced Artificial Intelligence*, part of its probabilistic-graphical-model and causal-AI block (course sessions 5–15). The session follows a fixed structure: a roadmap and learning objectives, the core theory organized in parts, worked numerical examples, and a part connecting the topic to intelligent agents, agentic AI, reinforcement learning, multi-agent systems and the other sessions of the course. Every number is reproduced by the companion Python scripts (`python/s06_examples.py` for the numbers, `python/s06_figures.py` for the figures), and `questions/s06_questions.tex` contains ten questions with complete answers.



The Advanced Artificial Intelligence course. Filled circles mark the PGM and causal-AI sessions; this is session 6.

Contents

Session 6 Bayesian Networks: DAGs and Independence Encoding **1**

6.1 From Chain Rule to Graph . . . 2
6.2 Compact Conditional Probability Distributions . . . 4
6.3 Reasoning Patterns in a Bayesian Network . . . 5
6.4 Local Markov Property and Factorization . . . 7
6.5 d-Separation . . . 7
6.6 The Markov Blanket . . . 10
6.7 I-Maps, Minimal I-Maps and Perfect Maps . . . 11
6.8 Markov Equivalence . . . 12
6.9 Bayesian Networks as World Models for Agents . . . 13
6.10 Summary . . . 15

SESSION 6 ADVANCED ARTIFICIAL INTELLIGENCE

Bayesian Networks: DAGs and Independence Encoding

DAG structure, joint distributions, conditional probability tables, d-separation and conditional independence

Advanced Artificial Intelligence · Probabilistic Graphical Models and Causal Inference

Instructor: Prof. M. Arashi

PART I
Representation
DAG + CPTs = joint distribution; compact CPDs; reasoning patterns.

PART II
Independence
Local Markov, d-separation, Bayes ball, Markov blanket.

PART III
Maps & Equivalence
I-maps, perfect maps, faithfulness, Markov equivalence.

PART IV
AI Connections
World models, tool-using agents, DBNs for RL, multi-agent BNs.

LEARNING OBJECTIVES

- Define a Bayesian network, write its chain-rule factorization and compute joint and conditional probabilities from its CPTs.
- Count free parameters and choose compact CPDs (deterministic, noisy-OR, tree, logistic, linear-Gaussian) when tables explode.
- Decide any query $X \perp\!\!\!\perp_d Y \mid Z$ by tracing chains, forks and colliders, and know that it can be decided in linear time (Bayes ball).
- Prove that factorization $\Leftrightarrow$ local Markov property; derive the Markov blanket; distinguish I-maps, minimal I-maps and perfect maps, and test Markov equivalence.
- Explain how BNs serve as world models for agents, as transition models in RL, and as distributed models in multi-agent systems.

Where we are. Session 5 gave us probability, conditional independence (CI) and the observation that a joint distribution is *compact* exactly when it factorizes. This session turns that observation into a modelling language, the *Bayesian network*. Sessions 7–10 compute with it, contrast it with undirected models and learn its CPTs; Sessions 12–15 read the same DAGs causally.

OPENING MOTIVATION

Conditional independence (CI) makes a joint distribution compact, but deriving CI statements algebraically does not scale to fifty variables. We need a *representation* we can draw, from which every CI statement the model assumes can be read off by inspection. Today's answer

is the Bayesian network: a directed acyclic graph (DAG) whose missing edges silently encode a whole family of conditional independencies, and a graphical test, d-separation, that lets us read them off at a glance.

PART I

Structure, Joint Distribution and CPTs

A graph for the qualitative assumptions, local tables for the numbers

PART I · REPRESENTATION

6.1 From Chain Rule to Graph

Recall the (always valid but generally useless) chain-rule factorization $P(x_1, \dots, x_n) = \prod_i P(x_i \mid x_1, \dots, x_{i-1})$. A Bayesian network is what you get when you are willing to *drop* some of the conditioning variables in each factor — keeping only a small, chosen subset $\mathrm{pa}(X_i)$ (the *parents* of X_i) for each variable, and asserting that this is enough.

Notation $\mathrm{pa}(X)$: parents $\mathrm{ch}(X)$: children $\mathrm{an}(X)$: ancestors $\mathrm{de}(X)$: descendants $\mathrm{nd}(X)$: non-descendants $\mathrm{MB}(X)$: Markov blanket

A *directed acyclic graph* has no directed cycle, so its nodes admit at least one *topological order* in which every parent precedes its children. A *trail* (or path) is a sequence of nodes joined by edges of either orientation; a *directed path* follows the arrows.

Definition 6.1 (Bayesian network)**.** A Bayesian network is a pair (G, Θ) where G is a directed acyclic graph (DAG) with nodes $X_1, \dots, X_n$, and Θ specifies a conditional probability distribution $P(x_i \mid \mathrm{pa}(x_i))$ for every node. The network defines a joint distribution via the **chain rule for Bayesian networks**:

$$P(x_1, \dots, x_n) = \prod_{i=1}^{n} P\big(x_i \mid \mathrm{pa}(x_i)\big). \tag{6.1}$$

The right-hand side of (6.1) is automatically a probability distribution: summing out the variables in reverse topological order, each local CPD sums to one in turn. This factorization is a genuine assumption, called the **local Markov assumption**: each variable is conditionally independent of its non-descendants given its parents, $X_i \perp\!\!\!\perp \mathrm{nd}(X_i) \mid \mathrm{pa}(X_i)$. The DAG's edges are exactly the conditioning relationships the modeler chose to keep; every *missing* edge is an implicit claim of irrelevance. (Theorem 6.2 below proves that "factorizes" and "satisfies the local Markov assumption" are the same thing.)

Worked Example 6.1 (The student-grades network)**.** Consider five variables familiar from any classroom: course **D**ifficulty (easy/hard), student **I**ntelligence (low/high), the **G**rade earned, the student's **S**AT score, and the recommendation **L**etter quality. A modeler encodes these causal-looking assumptions: Grade depends on Difficulty and Intelligence; SAT depends

only on Intelligence; the Letter depends only on the Grade. Figure 6.1 draws this. The chain rule for this network gives

$$P(d, i, g, s, l) = P(d)\,P(i)\,P(g \mid i, d)\,P(s \mid i)\,P(l \mid g),$$

five small factors with 15 free parameters in total (counted in (6.2) below) instead of one table over all $2 \times 2 \times 3 \times 2 \times 2 = 48$ joint configurations (G has three grade levels, the other variables two), which would need $48 - 1 = 47$ free numbers.

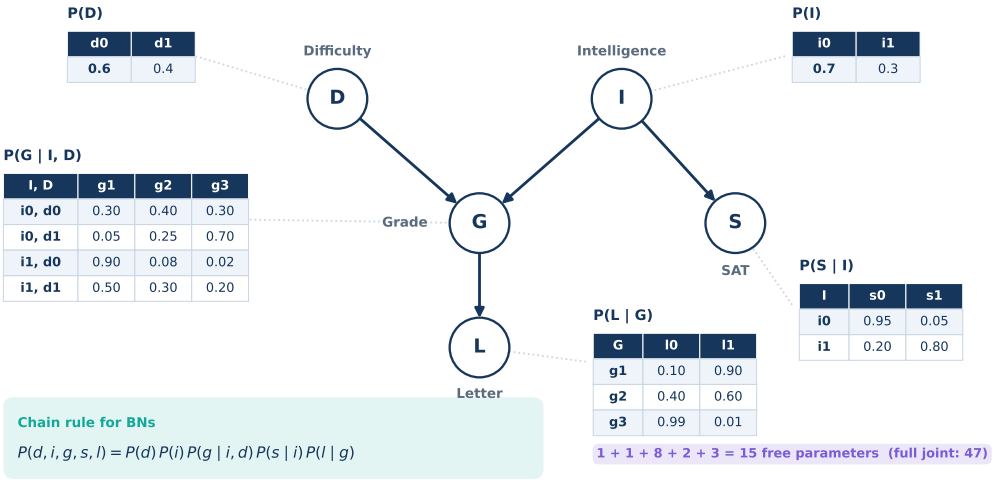


Figure 6.1. The student-grades Bayesian network with the conditional probability tables of Koller & Friedman (2009). Missing edges (e.g. no edge from D to S) are the model's independence assumptions; the five tables are all the numbers the model needs.

6.1.1 Conditional probability tables and a joint probability

A *conditional probability table* (CPT) lists, for every configuration u of the parents, a distribution over the child: each *row* sums to one. Figure 6.1 gives the full CPTs we use throughout the session. Values are coded as d^0=easy, d^1=hard; i^0=low, i^1=high; g^1=A, g^2=B, g^3=C; s^0=low, s^1=high SAT; l^0=weak, l^1=strong letter. For reference, the CPTs are $P(d^1) = 0.4$, $P(i^1) = 0.3$; $P(s^1 \mid i^0) = 0.05$, $P(s^1 \mid i^1) = 0.8$; $P(l^1 \mid g^1, g^2, g^3) = 0.9, 0.6, 0.01$; and $P(g^1, g^2, g^3 \mid i, d)$ equals $(0.3, 0.4, 0.3)$ for (i^0, d^0), $(0.05, 0.25, 0.7)$ for (i^0, d^1), $(0.9, 0.08, 0.02)$ for (i^1, d^0) and $(0.5, 0.3, 0.2)$ for (i^1, d^1).

Worked Example 6.2 (One entry of the joint)**.** Compute the probability that the course is easy, the student is intelligent, gets a B, has a high SAT score and still receives a weak letter.
Solution. Read one entry from each CPT and multiply, as (6.1) instructs:

$$\begin{aligned} P(d^0, i^1, g^2, s^1, l^0) &= P(d^0)P(i^1)P(g^2 \mid i^1, d^0)P(s^1 \mid i^1)P(l^0 \mid g^2) \\ &= 0.6 \cdot 0.3 \cdot 0.08 \cdot 0.8 \cdot 0.4 = 0.18 \cdot 0.08 \cdot 0.8 \cdot 0.4 = 0.0144 \cdot 0.32 = 0.004608. \end{aligned}$$

Summing all 48 such products gives exactly 1 (each CPT row sums to one). Scanning all 48 products (done by the script below), the single most probable configuration is $(d^1, i^0, g^3, s^0, l^0)$, a hard course, a less intelligent student, a C and a weak letter, with probability $P(d^1)P(i^0)P(g^3 \mid i^0, d^1)P(s^0 \mid i^0)P(l^0 \mid g^3) = 0.4 \cdot 0.7 \cdot 0.70 \cdot 0.95 \cdot 0.99 = 0.184338 \approx 0.1843$.

PYTHON LAB · s06_examples.py · ex_student_joint()

```
P(d0,i1,g2,s1,l0) = 0.004608; sum over all 48 joint entries = 1.000000
free parameters:  BN = 15, full joint = 47; most probable configuration p = 0.1843
its factors P(d1)P(i0)P(g3|i0,d1)P(s0|i0)P(l0|g3) = 0.40*0.70*0.70*0.95*0.99 = 0.184338
```

Parameter counting. A CPT for a child with k_i values and parents with k_j values has $\prod_{j\in\mathrm{pa}(i)} k_j$ rows, each with $k_i - 1$ free numbers, so the network has

$$\dim(G) = \sum_{i=1}^{n} (k_i - 1) \prod_{j\in\mathrm{pa}(i)} k_j \tag{6.2}$$

free parameters. For the student network: $P(D)$ and $P(I)$ need 1 each, $P(G \mid I, D)$ has 4 rows of 2 free numbers (8), $P(S \mid I)$ needs 2 and $P(L \mid G)$ needs 3: $1 + 1 + 8 + 2 + 3 = 15$ in total, against $48 - 1 = 47$ for the unrestricted joint. With n binary variables and in-degree at most 2, each node contributes at most $(2-1)\cdot 2^2 = 4$, so $\dim(G) \le 4n$: linear rather than exponential. For example, if the first node has no parent, the second one parent and every other node two parents, $\dim(G) = 1+2+4(n-2) = 4n-5$; for $n = 30$ this is 115, against $2^{30}-1 = 1{,}073{,}741{,}823 \approx 1.07\times 10^9$ for the full joint (`s06_examples.py`, function `ex_param_scaling()`).

PART I · REPRESENTATION

6.2 Compact Conditional Probability Distributions

A tabular CPT with n binary parents has 2^n rows. Ten causes of a symptom already need 1024 numbers, few of which an expert can elicit or data can estimate. The remedy is a *structured* CPD: a parametric family for $P(x \mid u)$ with far fewer parameters.

Definition 6.2 (Noisy-OR CPD)**.** Let Y have binary parents $X_1, \dots, X_n$. Each active parent $(X_i = 1)$ independently succeeds in turning Y on with probability λ_i, and a *leak* turns Y on with probability λ_0 even when no parent is active. Then

$$P(Y = 0 \mid x_1, \dots, x_n) = (1 - \lambda_0) \prod_{i=1}^{n} (1 - \lambda_i)^{x_i}, \qquad P(Y = 1 \mid \mathbf{x}) = 1 - P(Y = 0 \mid \mathbf{x}). \tag{6.3}$$

The derivation is a small BN in itself: introduce hidden "activation" variables $A_i \sim \mathrm{Bern}(\lambda_i)$, independent, and let $Y = A_0 \vee \bigvee_i (X_i \wedge A_i)$. Then $Y = 0$ iff every active cause failed and the leak failed, which is the product (6.3). The model has $n + 1$ parameters instead of 2^n, and each has a direct meaning that an expert can provide.

Worked Example 6.3 (Noisy-OR: cough, cold, flu, allergy)**.** Illustrative numbers: leak $\lambda_0 = 0.05$, cold $\lambda_1 = 0.5$, flu $\lambda_2 = 0.7$, allergy $\lambda_3 = 0.3$; Find $P(\text{cough} \mid \text{cold}, \text{flu}, \text{no allergy})$ and $P(\text{cough} \mid \text{all three causes})$.
Solution. By (6.3), $P(\text{cough} \mid \mathbf{x} = (1,1,0)) = 1-(1-0.05)(1-0.5)(1-0.7) = 1-0.95\cdot 0.5\cdot 0.3 = 1 - 0.1425 = 0.8575$; with all three causes, $1 - 0.1425 \cdot (1 - 0.3) = 1 - 0.09975 = 0.90025$. Each additional active cause multiplies the probability that the cough stays off by $1 - \lambda_i$.

PYTHON LAB · s06_examples.py · ex_noisy_or()

```
x=(1, 1, 0):  P(cough=1|x) = 0.85750; x=(1, 1, 1):  P(cough=1|x) = 0.90025
```

Other compact families (Table 6.1) follow the same idea: a *deterministic* CPD puts all mass on $y = f(u)$ (logical rules); a *tree-CPD* tests parents one at a time and stores distributions only at the leaves (context-specific independence); a *logistic CPD* sets $P(Y = 1 \mid \mathbf{u}) = \sigma(w_0 + \mathbf{w}^\top \mathbf{u})$. For continuous variables, the *linear-Gaussian CPD* is

$$X_i \mid \mathrm{pa}(X_i) = \mathbf{u} \;\sim\; \mathcal{N}\big(\beta_{i0} + \textstyle\sum_{j \in \mathrm{pa}(i)} b_{ij} u_j,\; d_i\big), \tag{6.4}$$

collected over all nodes below. Replacing the linear map by a neural network gives the *neural CPDs* of deep generative models (Part IV).

Table 6.1. Compact CPDs for a child with n parents. "Params" counts free parameters for binary variables (or per child for continuous ones).

CPD	params	assumption	typical use
Tabular	2^n	none	small parent sets
Deterministic	0	$y = f(u)$ exactly	logic, bookkeeping variables
Noisy-OR	$n + 1$	causes act independently	diagnosis, failure analysis
Tree-CPD	#leaves	context-specific independence	expert systems, sparse rules
Logistic	$n + 1$	additive effect on log-odds	discriminative-style CPDs
Linear-Gaussian	$n + 2$	linear mean, constant variance	sensor fusion, Kalman models
Neural	weights	flexible mean / parameters	VAEs, autoregressive models

Linear-Gaussian networks. Order the nodes topologically and collect the linear-Gaussian CPDs (6.4) of all nodes into one vector equation: with $B_{ij} = b_{ij}$ if $X_j \to X_i$ and 0 otherwise (B strictly lower triangular),

$$\mathbf{X} = \boldsymbol{\beta}_0 + B\mathbf{X} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, D), \quad D = \mathrm{diag}(d_1, \ldots, d_n). \tag{6.5}$$

Theorem 6.1 (A linear-Gaussian BN is a multivariate normal)**.** *Under* (6.5), $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ *with* $\boldsymbol{\mu} = (I - B)^{-1}\boldsymbol{\beta}_0$ *and* $\Sigma = (I - B)^{-1} D\, (I - B)^{-\top}$.

Proof. $I - B$ is unit lower triangular, so $\det(I - B) = 1$ and it is invertible; solving (6.5) gives $\mathbf{X} = (I - B)^{-1}(\boldsymbol{\beta}_0 + \boldsymbol{\varepsilon})$, an affine map of a Gaussian vector, hence Gaussian, with mean $(I - B)^{-1}\boldsymbol{\beta}_0$ (since $\mathbb{E}\boldsymbol{\varepsilon} = \mathbf{0}$) and covariance $(I - B)^{-1}\mathrm{Cov}(\boldsymbol{\varepsilon})(I - B)^{-\top}$. □

PART I · REPRESENTATION

6.3 Reasoning Patterns in a Bayesian Network

Before formalizing independence, watch how beliefs move through the student network (Fig. 6.2). Every number below is a ratio of sums of entries of the 48-entry joint (brute-force enumeration;

efficient algorithms are Session 7's subject), and each can be checked by hand from the CPTs. Three intermediate tables do most of the work, each obtained by summing out the parents of G:

$$P(G \mid i^0) = 0.6\,(0.3, 0.4, 0.3) + 0.4\,(0.05, 0.25, 0.7) = (0.20,\ 0.34,\ 0.46),$$
$$P(G \mid i^1) = 0.6\,(0.9, 0.08, 0.02) + 0.4\,(0.5, 0.3, 0.2) = (0.74,\ 0.168,\ 0.092),$$
$$P(G) = 0.7\,P(G \mid i^0) + 0.3\,P(G \mid i^1) = (0.362,\ 0.2884,\ 0.3496).$$

Causal reasoning (causes → effects). $P(l^1) = \sum_g P(g)P(l^1 \mid g) = 0.362 \cdot 0.9 + 0.2884 \cdot 0.6 + 0.3496 \cdot 0.01 = 0.502$; for a less intelligent student $P(l^1 \mid i^0) = 0.20 \cdot 0.9 + 0.34 \cdot 0.6 + 0.46 \cdot 0.01 = 0.389$; if the course is also easy, $P(l^1 \mid i^0, d^0) = 0.3 \cdot 0.9 + 0.4 \cdot 0.6 + 0.3 \cdot 0.01 = 0.513$.

Evidential reasoning (effects → causes). A C grade lowers intelligence from its prior 0.3 to $P(i^1 \mid g^3) = 0.3 \cdot 0.092/0.3496 = 0.079$. A weak letter alone: $P(l^0 \mid i^1) = 0.74 \cdot 0.1 + 0.168 \cdot 0.4 + 0.092 \cdot 0.99 = 0.23228$ and $P(l^0 \mid i^0) = 0.20 \cdot 0.1 + 0.34 \cdot 0.4 + 0.46 \cdot 0.99 = 0.6114$, so $P(i^1 \mid l^0) = \frac{0.3 \cdot 0.23228}{0.3 \cdot 0.23228 + 0.7 \cdot 0.6114} = \frac{0.069684}{0.497664} = 0.140$. Once the grade is known the letter adds nothing, $P(i^1 \mid l^0, g^3) = 0.079 = P(i^1 \mid g^3)$, because G blocks the chain $I \to G \to L$. Two effects of I can pull in opposite directions: combining the C with a high SAT score, $P(i^1 \mid g^3, s^1) = \frac{0.3 \cdot 0.092 \cdot 0.8}{0.3 \cdot 0.092 \cdot 0.8 + 0.7 \cdot 0.46 \cdot 0.05} = \frac{0.02208}{0.03818} = 0.578$.

Intercausal reasoning (between causes of a common effect). Given the C, learning that the course was hard raises intelligence from 0.079 to $P(i^1 \mid g^3, d^1) = \frac{0.3 \cdot 0.2}{0.3 \cdot 0.2 + 0.7 \cdot 0.7} = \frac{0.06}{0.55} = 0.109$: the difficulty partially *explains away* the poor grade. Symmetrically, given the C, a high SAT score (evidence for high intelligence) makes a hard course the more likely explanation: $P(d^1 \mid g^3) = 0.629$ rises to $P(d^1 \mid g^3, s^1) = 0.760$ (computed step by step in Worked Example 6.5). Given an A instead, a hard course makes intelligence *more* likely: $P(i^1 \mid g^1) = 0.3 \cdot 0.74/0.362 = 0.613$ but $P(i^1 \mid g^1, d^1) = \frac{0.3 \cdot 0.5}{0.3 \cdot 0.5 + 0.7 \cdot 0.05} = \frac{0.15}{0.185} = 0.811$.

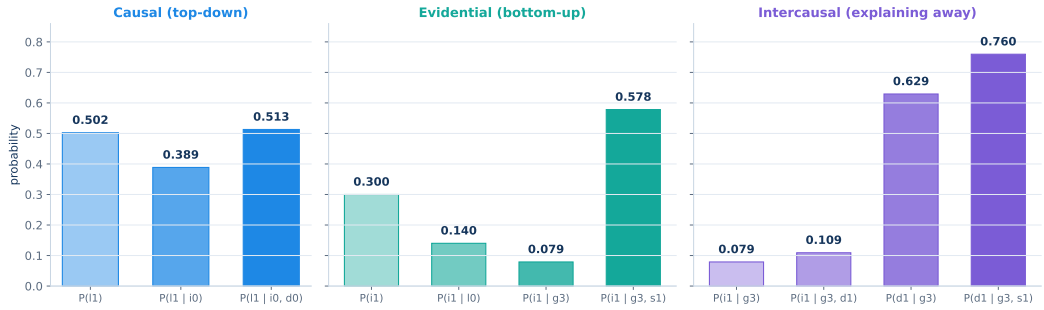


Figure 6.2. Three reasoning patterns in the student network, computed from the CPTs of Fig. 6.1 (the values derived in Section 6.3). Intercausal reasoning is the signature of a collider: D and I are marginally independent but become dependent once their common effect G is observed.

PYTHON LAB · s06_examples.py · ex_reasoning_patterns()

```
P(G) = [0.362, 0.2884, 0.3496]; P(G | i0) = [0.2, 0.34, 0.46]; P(G | i1) = [0.74, 0.168,
0.092]
P(l1)=0.502 P(l1|i0)=0.389 P(l1|i0,d0)=0.513 | P(i1|l0)=0.140 P(i1|g3)=0.079
P(i1|l0,g3)=0.079
P(i1|g3,s1)=0.578 | P(i1|g3,d1)=0.109 P(d1|g3)=0.629 P(d1|g3,s1)=0.760 P(i1|g1)=0.613
P(i1|g1,d1)=0.811
```

Influence flows along the graph in both directions; the only question is which paths are open, and that is exactly what d-separation answers.

PART II

Reading Independence off the Graph

Local Markov property, d-separation, Bayes ball and the Markov blanket

PART II · INDEPENDENCE

6.4 Local Markov Property and Factorization

Theorem 6.2 (Factorization ⇔ local Markov property)**.** *Let G be a DAG over $X_1, \dots, X_n$ and P a distribution over the same variables. Then P factorizes as in (6.1) if and only if P satisfies the local independencies*

$$\mathcal{I}_\ell(G) = \{\, X_i \perp\!\!\!\perp \mathrm{nd}(X_i) \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i) \;:\; i = 1, \dots, n \,\}.$$

Proof. ($\Leftarrow$) Fix a topological order $X_1, \dots, X_n$. The general chain rule gives $P(\mathbf{x}) = \prod_i P(x_i \mid x_1, \dots, x_{i-1})$. In a topological order the predecessors of X_i are non-descendants of X_i and include $\mathrm{pa}(X_i)$. The local independence plus the decomposition property (Session 5) give $X_i \perp\!\!\!\perp \{X_1, \dots, X_{i-1}\} \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i)$, so $P(x_i \mid x_1, \dots, x_{i-1}) = P(x_i \mid \mathrm{pa}(x_i))$.

($\Rightarrow$) Suppose (6.1) holds. Fix X_i; write $N = \mathrm{nd}(X_i)$ and $D = \mathrm{de}(X_i) \setminus \{X_i\}$. The set N is ancestral (a parent of a non-descendant is a non-descendant, otherwise the child would be a descendant), so there is a topological order listing N first, then X_i, then D. Sum the joint over D in reverse order: the last variable appears only in its own CPD, which sums to one; repeat. Hence $P(x_i, \mathbf{n}) = P(x_i \mid \mathrm{pa}(x_i)) \prod_{j \in N} P(x_j \mid \mathrm{pa}(x_j))$, and summing out x_i gives $P(\mathbf{n}) = \prod_{j \in N} P(x_j \mid \mathrm{pa}(x_j))$. Dividing, $P(x_i \mid \mathbf{n}) = P(x_i \mid \mathrm{pa}(x_i))$, which depends on $\mathbf{n}$ only through $\mathrm{pa}(x_i) \subseteq N$. That is $X_i \perp\!\!\!\perp N \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i)$. □

The local Markov assumption directly gives us *some* independencies (each node from its non-descendants given its parents), but it does not obviously tell us about, say, $D \perp\!\!\!\perp S$ (are course difficulty and SAT score independent, unconditionally?). We need a general graphical procedure that answers *any* query of the form "is $X \perp\!\!\!\perp Y \mid Z$ implied by this graph?" The key is to understand how probabilistic influence flows along paths, one edge-triple at a time.

PART II · INDEPENDENCE

6.5 d-Separation

6.5.1 Three building blocks

Every path in a DAG is built from three possible local patterns at each intermediate node, shown in Fig. 6.3.

- **Chain** $X \to Z \to Y$: X influences Y only *through* Z. If we observe Z, that channel is cut: $X \perp\!\!\!\perp Y \mid Z$. If Z is unobserved, influence flows freely.

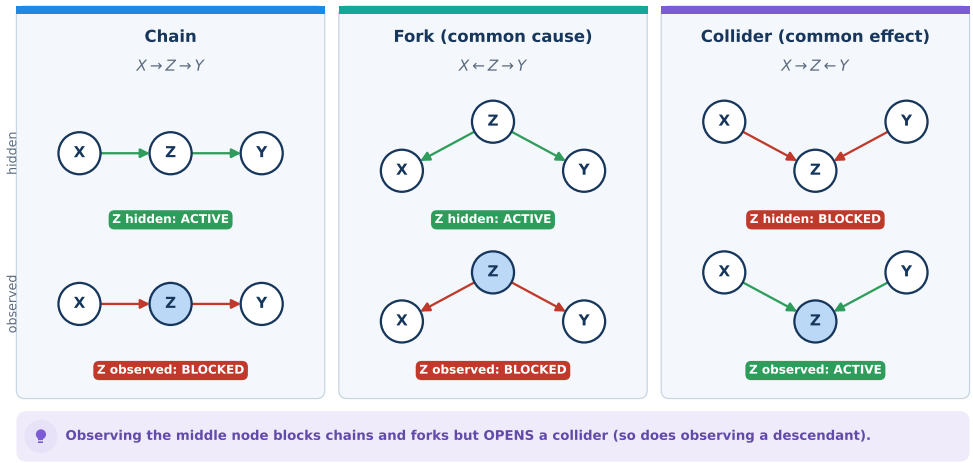


Figure 6.3. The three canonical connection patterns. Chains and forks transmit influence unless the middle node is observed; colliders do the opposite.

- **Fork** $X \leftarrow Z \rightarrow Y$: Z is a common cause; it creates a spurious correlation between X and Y. Again, conditioning on Z blocks the path: $X \perp\!\!\!\perp Y \mid Z$.
- **Collider** $X \rightarrow Z \leftarrow Y$: X and Y are marginally independent *causes* of a common effect Z — exactly like Session 5's XOR example, $Z = X \oplus Y$ with X, Y independent fair coins. (Session 5's enthusiast example is the opposite pattern, a fork: the enthusiast indicator is a common cause of the two ratings.) Here the logic flips: with Z unobserved, the path is blocked and $X \perp\!\!\!\perp Y$; but observing Z (or any descendant of Z) *opens* the path, inducing dependence ("explaining away").

Definition 6.3 (d-separation)**.** A path between X and Y is *blocked* by a set of nodes Z if it contains a chain or fork node in Z, or a collider node such that neither the collider nor any of its descendants is in Z. Sets X and Y are *d-separated* by Z, written $X \perp\!\!\!\perp_d Y \mid Z$, if Z blocks *every* path between them; a path that is not blocked is *active*, and if some path is active, X and Y are *d-connected* given Z. We write $\mathcal{I}(G) = \{(X \perp\!\!\!\perp Y \mid Z) : X \perp\!\!\!\perp_d Y \mid Z\}$ for the set of all independencies the graph asserts.

"The collider or one of its descendants is in Z" is the same as "the collider is in $\mathrm{an}(Z) \cup Z$". d-connection means only that the graph does *not* assert independence; whether the variables are actually dependent depends on the CPT numbers (see faithfulness, Section 6.7.1). In the worked examples we write $X \not\perp\!\!\!\perp Y \mid Z$ for a d-connected pair in this sense: dependent for generic CPTs, and dependent for the CPTs at hand whenever a number is given.

Theorem 6.3 (Soundness of d-separation)**.** *If $X \perp\!\!\!\perp_d Y \mid Z$ in DAG G, then $X \perp\!\!\!\perp Y \mid Z$ in* every *distribution P that factorizes according to G.*

The proof (Koller & Friedman 2009, Thm. 3.3) reduces the directed problem to an undirected one: keep only X, Y, Z and their ancestors, "marry" parents that share a child, drop the arrows, and show that (i) the distribution of the kept variables factorizes over this *moral ancestral graph*, (ii) d-separation in G implies ordinary graph separation there, and (iii) graph separation plus factorization implies CI by Session 5's factorized form of CI. A converse also holds — if X and Y are *not* d-separated given Z, then $X \not\perp\!\!\!\perp Y \mid Z$ for *almost every* choice of numerical parameters Θ consistent with G (Koller & Friedman 2009, Ch. 3; the property is called *faithfulness*, Section 6.7.1);

we state this without proof, since only soundness (Theorem 6.3) is needed to trust that a graph never claims an independence that isn't really there.

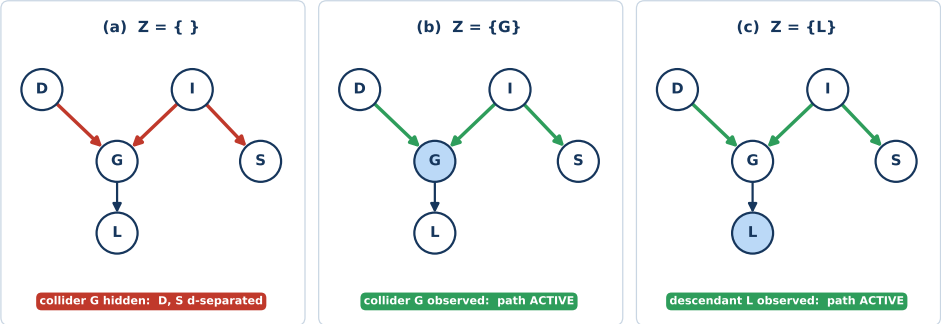


Figure 6.4. Observing the collider G (b), or its descendant L (c), opens the path $D \to G \leftarrow I \to S$ that is blocked when nothing is observed (a).

Worked Example 6.4 (Is $D \perp\!\!\!\perp S$?)**.** In the student network (Fig. 6.1), is Difficulty d-separated from SAT with no evidence, $Z = \emptyset$?

Solution. The only path between D and S is $D \to G \leftarrow I \to S$. The intermediate node G is a collider on this path (two arrowheads meet at G), and $G \notin Z = \emptyset$, and no descendant of G (namely L) is in Z either. An unobserved collider blocks a path, so this single path is blocked, and since it is the only path, $D \perp\!\!\!\perp_d S \mid \emptyset$. Difficulty and SAT score are independent — consistent with the intuitive causal story that they share no cause and no evidence has been observed to make them "compete" as explanations of anything. Numerically, summing out I (and G, L, whose CPT rows sum to one), $P(d^1, s^1) = P(d^1)\sum_i P(i)P(s^1 \mid i) = 0.4\,(0.7 \cdot 0.05 + 0.3 \cdot 0.8) = 0.4 \cdot 0.275 = 0.110$, which equals $P(d^1)P(s^1)$ because $P(s^1) = 0.275$.

Worked Example 6.5 (Conditioning on a collider creates dependence)**.** Now suppose we *do* observe the grade, $Z = \{G\}$. Is $D \perp\!\!\!\perp_d S \mid G$? See Fig. 6.4.

Solution. The same path $D \to G \leftarrow I \to S$ now has its collider G observed ($G \in Z$). An observed collider *opens* the path rather than blocking it. Since this is the only path and it is now active, $D \not\perp\!\!\!\perp S \mid G$. Intuitively: once you know a student got a C, the poor grade needs an explanation — a hard course or low intelligence. A high SAT score is evidence of high intelligence, which removes the second explanation and makes the first more likely; so S becomes informative about D. With the CPTs, summing over I (the L factor sums to one):

$$
\begin{aligned}
P(d^1, g^3) &= 0.4\,(0.7 \cdot 0.70 + 0.3 \cdot 0.20) = 0.4 \cdot 0.55 = 0.22, \\
P(d^0, g^3) &= 0.6\,(0.7 \cdot 0.30 + 0.3 \cdot 0.02) = 0.6 \cdot 0.216 = 0.1296, \\
P(d^1, g^3, s^1) &= 0.4\,(0.7 \cdot 0.70 \cdot 0.05 + 0.3 \cdot 0.20 \cdot 0.8) = 0.4 \cdot 0.0725 = 0.029, \\
P(d^0, g^3, s^1) &= 0.6\,(0.7 \cdot 0.30 \cdot 0.05 + 0.3 \cdot 0.02 \cdot 0.8) = 0.6 \cdot 0.0153 = 0.00918,
\end{aligned}
$$

so $P(d^1 \mid g^3) = 0.22/(0.22 + 0.1296) = 0.629$ but $P(d^1 \mid g^3, s^1) = 0.029/(0.029 + 0.00918) = 0.760$. This "explaining away" phenomenon is a direct generalization of Session 5's XOR example (conditioning on the common effect $Z = X \oplus Y$ made two independent coins dependent), and it is also the graphical seed of an important warning: naively "controlling for everything you can measure" can *create* spurious associations rather than remove them — a theme we revisit sharply in Session 12.

PYTHON LAB · s06_examples.py · ex_dsep_checks()

```
D dsep S | {}:  True; | {G}:  False; | {L}:  False; P(s1)=0.275 P(d1,s1)=0.110
P(d1)P(s1)=0.110
P(d1, g3) = 0.2200 P(d1, g3, s1) = 0.02900; P(d0, g3) = 0.1296 P(d0, g3, s1) = 0.00918
P(d1|g3)=0.629 but P(d1|g3,s1)=0.760
```

Deciding d-separation efficiently. Checking every path is exponential in general, because a DAG can have exponentially many trails. The *Bayes-ball* (reachability) algorithm instead searches over pairs (node, direction of arrival): a ball starts at X, and at every node the three building blocks decide whether it may continue up or down; colliders are passable only if they lie in Z or have a descendant in Z, a set computed first by one backward search from Z. Since each of the $2|V|$ pairs is expanded once, the whole test runs in time $O(|V| + |E|)$ (Koller & Friedman 2009, Alg. 3.1). The function `ex_bayes_ball()` implements it; on the student network it finds that from D the ball reaches $\{G, L\}$ with no evidence, $\{I, S\}$ given G, and $\{G, I, S\}$ given L.

PART II · INDEPENDENCE

6.6 The Markov Blanket

Definition 6.4 (Markov blanket)**.** The Markov blanket of X in a DAG is $\mathrm{MB}(X) = \mathrm{pa}(X) \cup \mathrm{ch}(X) \cup \big(\bigcup_{C \in \mathrm{ch}(X)} \mathrm{pa}(C)\big) \setminus \{X\}$: parents, children, and the children's other parents (co-parents). Co-parents are included because each child is a collider between X and its co-parent.

Theorem 6.4 (Markov blanket shields a node)**.** *Let $\mathcal{X} = \{X_1, \dots, X_n\}$ be the set of all variables and $\mathbf{x}_{-X}$ an assignment to all variables other than X. If P factorizes over G then for every node X, $X \perp\!\!\!\perp \mathcal{X} \setminus (\{X\} \cup \mathrm{MB}(X)) \mid \mathrm{MB}(X)$, and, wherever $P(\mathbf{x}_{-X}) > 0$,*

$$P(x \mid \mathbf{x}_{-X}) = \frac{P(x \mid \mathrm{pa}(x)) \prod_{C \in \mathrm{ch}(X)} P(c \mid \mathrm{pa}(c))}{\sum_{x'} P(x' \mid \mathrm{pa}(x)) \prod_{C \in \mathrm{ch}(X)} P(c \mid \mathrm{pa}(c))\big|_{x \to x'}}. \tag{6.6}$$

Proof. Write $P(x \mid \mathbf{x}_{-X}) = P(x, \mathbf{x}_{-X}) / \sum_{x'} P(x', \mathbf{x}_{-X})$ and substitute the factorization (6.1) in numerator and denominator. Every CPD that does not mention X — neither as child nor as parent — is a common factor and cancels. What remains is X's own CPD and the CPDs of its children, which is (6.6). These factors mention only X, $\mathrm{pa}(X)$, $\mathrm{ch}(X)$ and the parents of the children, i.e. X and $\mathrm{MB}(X)$. Hence the conditional depends on $\mathbf{x}_{-X}$ only through $\mathrm{MB}(X)$, which is the stated independence. □

Worked Example 6.6 (A Markov blanket in the student network)**.** In the student network (Fig. 6.1), $\mathrm{MB}(G) = \{D, I, L\}$: parents D, I, child L, and no co-parents (L has no other parent); $\mathrm{MB}(I) = \{D, G, S\}$: children G, S and the co-parent D of G. Compute $P(G \mid d^1, i^1, s^1, l^1)$. *Solution.* By (6.6), $P(g \mid d^1, i^1, s^1, l^1) \propto P(g \mid i^1, d^1)\, P(l^1 \mid g)$. The unnormalized values for g^1, g^2, g^3 are $0.5 \cdot 0.9 = 0.45$, $0.3 \cdot 0.6 = 0.18$ and $0.2 \cdot 0.01 = 0.002$, with sum 0.632; dividing, $P(G \mid \cdot) = (0.45, 0.18, 0.002)/0.632 = (0.712, 0.285, 0.003)$. The SAT score is outside the blanket, and indeed dropping s^1 from the evidence leaves the answer unchanged.

PYTHON LAB · s06_examples.py · ex_markov_blanket()

```
MB(G) = ['D','I','L'] MB(I) = ['D','G','S']; unnormalised [0.45 0.18 0.002], sum = 0.632
P(G | d1,i1,s1,l1):  enumeration [0.712 0.285 0.003], local MB formula [0.712 0.285 0.003]
P(G | d1,i1,l1) (S dropped) [0.712 0.285 0.003]
```

PART III

I-Maps, Perfect Maps and Markov Equivalence

What exactly does a graph promise about a distribution?

6.7 I-Maps, Minimal I-Maps and Perfect Maps

Let $\mathcal{I}(P)$ be the set of all CI statements that hold in P. A graph is useful if what it asserts is true; it is *maximally* useful if it asserts everything that is true.

Definition 6.5 (I-map, minimal I-map, perfect map)**.** A DAG G is an *I-map* of P if $\mathcal{I}(G) \subseteq \mathcal{I}(P)$. It is a *minimal I-map* if removing any edge destroys this property. It is a *perfect map* (P-map) if $\mathcal{I}(G) = \mathcal{I}(P)$.

By Theorems 6.2 and 6.3, G is an I-map of P if and only if P factorizes over G. The complete DAG (every pair joined) asserts nothing and is therefore an I-map of every distribution — a correct but useless model. Minimal I-maps are constructed greedily: fix an order $X_1, \dots, X_n$ and choose as $\mathrm{pa}(X_i)$ a smallest subset $U \subseteq \{X_1, \dots, X_{i-1}\}$ with $X_i \perp\!\!\!\perp \{X_1, \dots, X_{i-1}\} \setminus U \mid U$. By the ($\Leftarrow$) part of Theorem 6.2 the result is an I-map, and it is minimal by construction.

Worked Example 6.7 (The order of the variables matters)**.** Build minimal I-maps of the student distribution with two variable orders, assuming that the CI statements true in P are exactly those of the student DAG (faithfulness, Section 6.7.1), so that d-separation in Fig. 6.1 can serve as the CI oracle. Count parameters with (6.2) (G has 3 values, the others 2).
Solution. Order D, I, G, S, L ("causal"). D: no predecessors. I: $I \perp\!\!\!\perp D$, so no parent. G: no proper subset $U \subset \{D, I\}$ makes $G \perp\!\!\!\perp \{D, I\} \setminus U \mid U$, so $\mathrm{pa}(G) = \{D, I\}$. S: $S \perp\!\!\!\perp \{D, G\} \mid I$, so $\mathrm{pa}(S) = \{I\}$. L: $L \perp\!\!\!\perp \{D, I, S\} \mid G$, so $\mathrm{pa}(L) = \{G\}$. This is the original 4-edge graph with $1 + 1 + 8 + 2 + 3 = 15$ parameters.
Order L, S, G, I, D ("diagnostic"). The same procedure gives the 7 edges $L \to S$, $L \to G$, $S \to G$, $S \to I$, $G \to I$, $G \to D$, $I \to D$ and $1 + 2 + (2 \cdot 4) + (1 \cdot 6) + (1 \cdot 6) = 23$ parameters (for L; $S \mid L$; $G \mid L, S$; $I \mid S, G$; $D \mid G, I$).
Both are I-maps of the same P; only the first is a perfect map (the second has extra edges and therefore fail to assert some true CI statements, e.g. $D \perp\!\!\!\perp I$). Orders that follow causal direction tend to produce sparse graphs — an empirical observation that Sessions 12–13 will elevate to a principle.

PYTHON LAB · s06_examples.py · ex_minimal_imap()

```
order DIGSL: 4 edges, 15 params; order LSGID: 7 edges, 23 params
LSGID edges [(G,D),(G,I),(I,D),(L,G),(L,S),(S,G),(S,I)]
```

Not every distribution has a perfect map. Take Session 5's XOR example: X, Y fair independent coins and $Z = X \oplus Y$. Every pair is independent, yet the three are jointly dependent. A DAG asserting $X \perp\!\!\!\perp Y$, $X \perp\!\!\!\perp Z$ and $Y \perp\!\!\!\perp Z$ must have no edges at all (adjacent nodes are never d-separated), and the empty graph asserts mutual independence, which is false. So no DAG is a P-map; the best we can do is a minimal I-map such as $X \to Z \leftarrow Y$. Session 8 shows distributions (the four-cycle) whose natural graph is undirected.

6.7.1 Faithfulness

Definition 6.6 (Faithfulness)**.** P is *faithful* to G if G is a perfect map of P, $\mathcal{I}(G) = \mathcal{I}(P)$: P factorizes over G, and every CI that holds in P is implied by d-separation in G.

Faithfulness fails only through exact numerical coincidences: the unfaithful CPT values form a Lebesgue-null set (Koller & Friedman 2009, Ch. 3); in linear-Gaussian models, the zero set of a polynomial in the edge weights, e.g. when a direct effect $X \to Y$ is exactly cancelled by an indirect one $X \to M \to Y$, so that $X \perp\!\!\!\perp Y$ although $X \to Y$ is an edge (Session 13 works this out).

PART III · MAPS AND EQUIVALENCE

6.8 Markov Equivalence

Different DAGs can assert exactly the same independencies. $X \to Y \to Z$, $X \leftarrow Y \leftarrow Z$ and $X \leftarrow Y \to Z$ all assert $X \perp\!\!\!\perp Z \mid Y$ and nothing else, whereas $X \to Y \leftarrow Z$ asserts $X \perp\!\!\!\perp Z$ (Fig. 6.5). Counting all 25 DAGs on three labelled nodes: the empty graph (1 DAG) is a class of size 1; each of the 3 one-edge skeletons gives a class of 2 DAGs (both orientations); each of the 3 two-edge skeletons gives a class of size 3 (chain, reverse chain, fork) and a v-structure class of size 1; the 6 acyclic orientations of the triangle form one class. That is $1 + 6 + 12 + 6 = 25$ DAGs in only $1 + 3 + 6 + 1 = 11$ classes.

Definition 6.7 (Markov equivalence, v-structure)**.** G_1 and G_2 are *Markov equivalent* if $\mathcal{I}(G_1) = \mathcal{I}(G_2)$. The *skeleton* of G is its undirected version. A *v-structure* (immorality) is a triple $X \to Z \leftarrow Y$ with X and Y non-adjacent.

Theorem 6.5 (Verma–Pearl characterization)**.** *Two DAGs are Markov equivalent if and only if they have the same skeleton and the same v-structures.*

The idea of the proof (Verma & Pearl 1990; Koller & Friedman 2009, Thm. 3.8): adjacent nodes can never be d-separated, so the independencies determine the skeleton; and for a non-adjacent pair X, Y with a common neighbour Z, the middle node Z must be *excluded* from every separating set if $X \to Z \leftarrow Y$ is a v-structure but *included* in every one if it is a chain or fork, so the independencies also determine the v-structures.

An equivalence class is represented by a *CPDAG*: edges oriented alike in every member are directed,

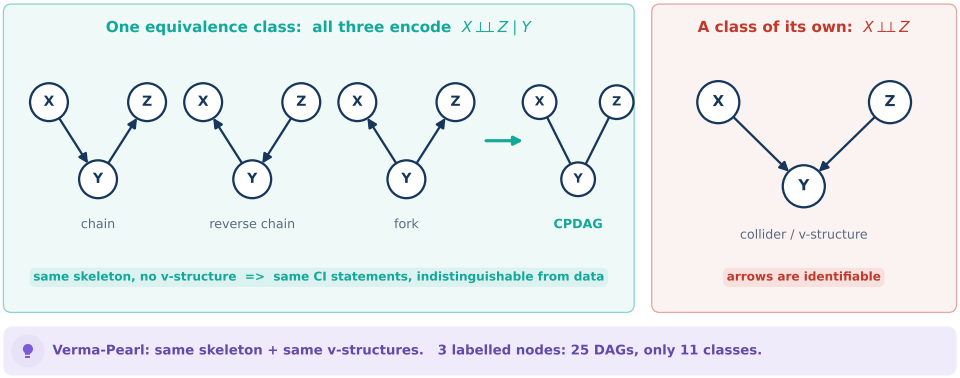


Figure 6.5. Markov equivalence. The chain, reverse chain and fork form one class, represented by an undirected CPDAG; the v-structure is alone in its class and its arrows can be recovered from observational data.

the rest undirected. Observational data identify only the CPDAG (Session 11); orienting the rest needs assumptions or interventions (Sessions 12–14).

Worked Example 6.8 (Analytical: which DAGs are equivalent?)**.** (a) Is the student network equivalent to the version with $S \to I$ instead of $I \to S$? (b) to the version with $G \to D$ instead of $D \to G$? (c) For the diamond $A \to B$, $A \to C$, $B \to D$, $C \to D$: is $B \to A$ allowed? is $D \to B$?

Solution. (a) Yes: the skeleton is unchanged and the only v-structure $D \to G \leftarrow I$ survives; S–I–G was a fork at I and becomes a chain $S \to I \to G$. (b) No: reversing $D \to G$ turns the v-structure $D \to G \leftarrow I$ into the chain $D \leftarrow G \leftarrow I$, so the graph no longer asserts $D \perp\!\!\!\perp I$ (it asserts $D \perp\!\!\!\perp I \mid G$ instead). (c) $B \to A$: skeleton and the v-structure $B \to D \leftarrow C$ are unchanged, so yes. $D \to B$: the v-structure at D disappears and a new one $A \to B \leftarrow D$ appears; not equivalent.

PYTHON LAB · s06_examples.py · ex_markov_equivalence()

```
chain~rev.chain True, chain~fork True, chain~collider False; student~(S->I) True, ~(G->D)
False
diamond~(B->A) True, ~(D->B) False; 3 labelled nodes:  25 DAGs, 11 classes, sizes
[1,1,1,1,2,2,2,3,3,3,6]
```

PART IV

Bayesian Networks in Agents, RL and Multi-Agent Systems

The same DAG is a world model, a transition model and a distributed belief

PART IV · AI CONNECTIONS

6.9 Bayesian Networks as World Models for Agents

A rational, model-based agent (course sessions 1 and 25) maps percepts to actions through an internal model of the unobserved world. A BN is the simplest explicit, inspectable such model: its

DAG says what affects what, its CPTs say how strongly, d-separation says which observations are relevant to which hidden variables, and updating the agent's state on a new percept is a posterior computation.

6.9.1 Agentic AI: which tool should the agent call next?

AI CONNECTION · Agentic AI

Tool-using LLM agents (course session 26) must decide *which* tool to call. When the uncertainty is over a few discrete causes, a small BN behind the agent turns this into a computation: score each tool by the expected reduction in uncertainty about the causes, call the best one, enter its result as evidence, repeat. The language model handles dialogue and tool syntax; the BN handles the probabilistic bookkeeping.

Worked Example 6.9 (Deployment-failure diagnosis (hypothetical numbers))**.** Causes: configuration error C (prior 0.30) and network outage N (prior 0.10), independent; symptom F = deployment fails, noisy-OR with $\lambda_0 = 0.02$, $\lambda_C = 0.8$, $\lambda_N = 0.9$. Tools: `read_logs` reports a config problem with sensitivity 0.90 and false-positive rate 0.10; `ping` reports an outage with sensitivity 0.95, false-positive rate 0.05. The user reports $F = 1$. Which tool first?

Each tool's report depends only on its own cause: $P(\text{logs}{=}1 \mid C{=}1) = 0.90$, $P(\text{logs}{=}1 \mid C{=}0) = 0.10$, $P(\text{ping}{=}1 \mid N{=}1) = 0.95$, $P(\text{ping}{=}1 \mid N{=}0) = 0.05$. Score a tool by its expected information gain $\text{IG} = H(C, N \mid F{=}1) - \sum_o P(o \mid F{=}1)\, H(C, N \mid F{=}1, o)$, with entropies in bits.

Solution. (1) Noisy-OR: $P(F{=}1 \mid c, n) = 1 - 0.98 \cdot 0.2^c \cdot 0.1^n$, i.e. 0.02, 0.902, 0.804, 0.9804 for $(c, n) = (0,0), (0,1), (1,0), (1,1)$. Multiplying by the priors $P(c)P(n)$ gives the joint weights $0.7 \cdot 0.9 \cdot 0.02 = 0.0126$, $0.7 \cdot 0.1 \cdot 0.902 = 0.06314$, $0.3 \cdot 0.9 \cdot 0.804 = 0.21708$, $0.3 \cdot 0.1 \cdot 0.9804 = 0.029412$, whose sum is $P(F{=}1) = 0.3222$. (2) Dividing, the posterior over (c, n) is (0.0391, 0.1959, 0.6737, 0.0913), so $P(C{=}1 \mid F) = 0.6737 + 0.0913 = 0.7650$, $P(N{=}1 \mid F) = 0.1959 + 0.0913 = 0.2872$, and $H(C, N \mid F{=}1) = -\sum p \log_2 p = 1.34277$ bits. (3) `read_logs`: $P(\text{logs}{=}1 \mid F) = 0.9 \cdot 0.7650 + 0.1 \cdot 0.2350 = 0.7120$; the posteriors after logs=1 and logs=0 (multiply each state by its likelihood 0.9/0.1 resp. 0.1/0.9 and renormalize) have entropies 0.7407 and 1.4520, so the expected entropy is $0.7120 \cdot 0.7407 + 0.2880 \cdot 1.4520 = 0.94559$ and $\text{IG} = 1.34277 - 0.94559 = 0.39718$ bits. (4) `ping`: $P(\text{ping}{=}1 \mid F) = 0.95 \cdot 0.2872 + 0.05 \cdot 0.7128 = 0.3085$; posterior entropies 1.3495 and 0.4648, expected entropy $0.3085 \cdot 1.3495 + 0.6915 \cdot 0.4648 = 0.73773$, $\text{IG} = 1.34277 - 0.73773 = 0.60504$ bits.

`ping` wins: although the config error is the likelier cause, `ping` is sharper and targets the more uncertain hypothesis. If `ping` reports an outage, the posterior becomes $(0.0063, 0.6034, 0.1092, 0.2811)$ and the config error is explained away: $P(C{=}1 \mid F, \text{ping}{=}1) = 0.1092 + 0.2811 = 0.3903$. Information gain is a utility-free proxy for the *value of information* (course session 21).

PYTHON LAB · s06_examples.py · ex_agent_voi()

```
joint P(c,n,F=1) = [0.0126, 0.06314, 0.21708, 0.029412]; P(F=1) = 0.3222
P(C=1|F=1) = 0.7650, P(N=1|F=1) = 0.2872; H(C,N|F=1) = 1.34277 bits
read_logs:  entropies 0.7407/1.4520, expected 0.94559, information gain = 0.39718 bits
ping:  entropies 1.3495/0.4648, expected 0.73773, information gain = 0.60504 bits
P(C=1 | F=1, ping=1) = 0.3903
```

6.9.2 Reinforcement learning: dynamic Bayesian networks

AI CONNECTION · Reinforcement Learning

An MDP's transition model $P(s_{t+1} \mid s_t, a_t)$ is a CPT. When the state is a vector of variables, a *dynamic Bayesian network* (DBN) represents it compactly: a two-slice template (2-TBN) gives each next-step variable a few parents in the current slice, $P(\mathbf{x}^{t+1} \mid \mathbf{x}^t, a^t) = \prod_i P(x_i^{t+1} \mid \mathrm{pa}(X_i^{t+1}))$, and unrolling the template over time gives the full BN. HMMs and POMDPs are the special cases with hidden states and observation CPDs $P(o_t \mid s_t)$. The Markov property itself is a d-separation statement: in the unrolled graph $S_{t+1} \perp\!\!\!\perp_d \{S_{0:t-1}, A_{0:t-1}\} \mid S_t, A_t$.

Worked Example 6.10 (Factored versus flat transition models)**.** A robot's state consists of $n = 10$ binary variables, each with at most 2 parents in the previous slice. A flat transition table needs $2^{10}(2^{10} - 1) = 1{,}047{,}552$ parameters per action; the DBN needs $10 \cdot 2^2 = 40$. With $n = 20$: about 1.1×10^{12} versus 80. Factored MDPs (course session 20) exploit this structure.

PYTHON LAB · s06_examples.py · ex_factored_mdp()

```
n=10:  flat 1,047,552 vs DBN 40 params per action; n=20:  flat 1,099,510,579,200 vs DBN 80
```

LINK TO COURSE SESSION 17 · RL and MDPs

$P(s' \mid s, a)$ and $R(s, a)$ are CPDs of a DBN with the action as a parent of the next state; course session 21 adds the observation CPD, and updating the resulting *belief state* is BN inference (Session 7).

6.9.3 Multi-agent AI: distributed Bayesian networks

AI CONNECTION · Multi-Agent AI

In multi-agent sensing each agent observes a different part of the world. In a *multiply-sectioned* BN, each agent owns a sub-network, and neighbouring agents share *interface* variables (e.g. a camera agent and a thermal-sensor agent both reasoning about *Fire*). If the interface between two agents d-separates their private variables, each agent can do inference locally and exchange only beliefs (messages) about interface variables, and the combined answer equals the centralized one — the multi-agent version of the Markov blanket.

PART IV · AI CONNECTIONS

6.10 Summary

THINK ABOUT IT **Two graphs encode the same independencies but disagree about the direction of an edge. Which observations, if any, could tell them apart?**

THINK ABOUT IT **Why are co-parents in the Markov blanket even though they are not adjacent to the node?**

Table 6.2. Cheat sheet of key formulas.

Object	Formula / rule
BN chain rule	$P(\mathbf{x}) = \prod_i P(x_i \mid \mathrm{pa}(x_i))$
Local Markov	$X_i \perp\!\!\!\perp \mathrm{nd}(X_i) \setminus \mathrm{pa}(X_i) \mid \mathrm{pa}(X_i) \;\Leftrightarrow$ factorization
Parameters	$\sum_i (k_i - 1) \prod_{j \in \mathrm{pa}(i)} k_j$
Noisy-OR	$P(Y = 0 \mid \mathbf{x}) = (1 - \lambda_0) \prod_i (1 - \lambda_i)^{x_i}$
d-separation	every path blocked: observed chain/fork node, or collider with no observed descendant
Markov blanket	$\mathrm{MB}(X) = \mathrm{pa} \cup \mathrm{ch} \cup \mathrm{pa}(\mathrm{ch})$; $P(x \mid \mathrm{rest}) \propto P(x \mid \mathrm{pa}) \prod_c P(c \mid \mathrm{pa}(c))$
Markov equivalence	same skeleton and same v-structures
Linear-Gaussian	$\Sigma = (I - B)^{-1} D (I - B)^{-\top}$

Further reading. Koller & Friedman (2009), *Probabilistic Graphical Models*, Ch. 3, 5 and 6; Bishop (2006), *Pattern Recognition and Machine Learning*, Ch. 8.1–8.2; Pearl (2009), *Causality*, Ch. 1.

CONNECTIONS AND TAKEAWAYS

We now have a representation (the DAG) that compactly encodes a factorization, and an algorithm (d-separation, in linear time by Bayes ball) that reads off every CI statement the graph implies without touching the numbers in Θ. But every posterior in this session, like $P(I = \text{high} \mid L = \text{weak})$, was computed by brute-force enumeration of the joint table, which is intractable for large networks. **Session 7** turns the graph's structure into algorithms — variable elimination, belief propagation and MAP inference — that compute exact answers without ever building the full joint table.